

# Sandalwood: A High-Performance Python Library for Multivariate Taylor Functions and Differential Algebra

Shashikant Manikonda<sup>1</sup>

<sup>1</sup>*Independent Researcher*

We introduce **sandalwood**, an open-source Python library designed for the creation, manipulation, and composition of Multivariate Taylor Functions (MTF). Built upon the mathematical foundations of Differential Algebra (DA) and Truncated Power Series Algebra (TPSA), **sandalwood** provides a powerful framework for extracting high-order derivatives and tracking nonlinear dynamics in complex systems. The package features a dual-backend architecture, seamlessly combining a Just-In-Time (JIT) compiled Python/Numba engine for portability with an optional high-performance Fortran backend based on COSY Infinity. This hybrid approach enables massive parallelization, optimized memory management via zero-copy data transfers, and efficient map composition. **sandalwood** offers a highly accessible, Pythonic API that is ideally suited for High-Performance Computing (HPC) workflows in fields such as electromagnetics and beam physics, as well as serving as a robust environment for autonomous AI-driven scientific simulation.

## I. INTRODUCTION

The analysis of complex dynamical systems often requires accurate representations of nonlinear behaviors that govern long-term stability. While ordinary differential equations (ODEs) can describe these systems, purely analytical solutions are frequently intractable. Traditional numerical integration methods, such as low-order Taylor expansions or Runge-Kutta schemes, compute system states point-by-point in phase space, but they often struggle to efficiently capture the intricate, high-order nonlinear effects required for long-term tracking.

To overcome these limitations, the field of Differential Algebra (DA), computationally implemented as Truncated Power Series Algebra (TPSA), offers a profound shift in perspective. Instead of propagating individual points through a system, DA operates on the algebra of functions, treating the truncated Taylor expansion of a function as a single, manipulable mathematical object. This allows derivatives to be propagated algorithmically, avoiding the combinatorial explosion associated with symbolic differentiation and enabling highly efficient high-order calculations.

**sandalwood** is an open-source Python package developed to bring the power of high-order Differential Algebra to the modern scientific computing ecosystem. While established tools like COSY INFINITY [3, 4] offer exceptional performance in Fortran, interfacing them with modern data science workflows, machine learning frameworks, and automated scientific pipelines presents significant challenges. **sandalwood** addresses this by providing an elegant, object-oriented Python API backed by a sophisticated hybrid execution engine. It is designed to meet the demands of High-Performance Computing (HPC) in nonlinear dynamics, offering advanced features like parallel map evaluation, batch composition, and deep integration with numerical libraries.

## II. THEORETICAL BACKGROUND

Differential algebra is formally concerned with the study of differential equations and operators as algebraic structures, such as differential rings and fields [1]. For computational applications, this mathematical framework is realized as Truncated Power Series Algebra (TPSA) [2]. The core principle is that the local behavior of a sufficiently smooth function can be completely described by its truncated Taylor expansion, referred to as a DA vector.

A DA vector, denoted as  $[f]_n$ , represents the Taylor expansion of a function  $f$  at a specific expansion point  $x_0$  up to a given order  $n$ :

$$[f]_n = \sum C_{n_1, \dots, n_v} \cdot d_1^{n_1} \cdot \dots \cdot d_v^{n_v} \quad (1)$$

where  $v$  is the number of variables,  $d_i$  represent small variances in the input variables, and the coefficients  $C$  correspond to the scaled partial derivatives of the function at the expansion point.

By defining operations such as addition, multiplication, and differentiation on these vectors, TPSA allows us to compute the DA vector of complex composite functions. When a mathematical operation is applied to a DA vector, the result is a new DA vector containing the correct Taylor expansion of the resulting function.

Programmatically, **sandalwood** represents these mathematical constructs using two primary data types: dense and sparse representations. The dense approach stores all coefficients in a contiguous block of memory, ensuring fast SIMD-vectorized operations at lower orders. However, as the number of terms grows polynomially as  $\binom{v+n}{n}$ , the sparse representation—storing only non-zero coefficients—becomes a computational necessity for high-order work, exploiting the inherent symmetries and linearities present in many physical systems.

### III. SOFTWARE ARCHITECTURE

**sandalwood** is designed around an intuitive object-oriented API that abstracts the underlying computational complexity.

#### A. Object-Oriented Design

The library exposes two core classes:

- **MultivariateTaylorFunction (MTF)**: Represents a scalar-valued function of multiple variables. It encapsulates the DA vector and provides overloaded arithmetic operators (e.g., `+`, `*`, `**`) and transcendental functions (e.g., `sin`, `exp`).
- **TaylorMap**: Represents a vector-valued function, essentially a collection of **MultivariateTaylorFunction** components. It is the primary structure for studying coordinate transformations, systems of equations, and the flow of differential equations.

#### B. Hybrid Execution Engine

To overcome the overhead commonly associated with Python, **sandalwood** employs a Layered Hybrid Strategy:

- **Native Python/Numba Backend**: The default backend utilizes Numba to Just-In-Time (JIT) compile dense algebraic kernels directly into machine code. It implements block-based reduction strategies and SIMD vectorization, processing evaluations in localized memory blocks to maximize CPU cache hit rates. This backend is highly portable and requires no compilation step.
- **COSY Infinity Fortran Backend**: For massive batch operations or extremely high-order compositions, **sandalwood** can seamlessly dispatch calculations to a compiled Fortran core based on COSY Infinity. This is achieved via a Direct Memory Bridge using `ctypes` and Fortran’s `ISO_C_BINDING`, allowing Python to manipulate Fortran-allocated static memory directly without serialization overhead.

### IV. CORE FEATURES AND USAGE

**sandalwood** is designed to be highly accessible. The following code snippet demonstrates the initialization of variables, definition of a Taylor series, and evaluation of a non-linear map.

```
1 from sandalwood import
   MultivariateTaylorFunction, TaylorMap, Var
2
3 # Initialize the global MTF environment
```

```
4 # 2 variables (x, y), up to order 3
5 MultivariateTaylorFunction.initialize_mtf(
   max_order=3, max_dimension=2)
6
7 # Define symbolic variables
8 x, y = Var(1), Var(2)
9
10 # Define a non-linear map F(x, y) = [x + y**2, y
   - x**2]
11 F = TaylorMap([x + y**2, y - x**2])
12
13 # Create another map G(a, b) = [a*b, a]
14 a, b = Var(1), Var(2)
15 G = TaylorMap([a * b, a])
16
17 # Compose the maps: H(x, y) = G(F(x, y))
18 H = G.compose(F)
19
20 # Compute the inverse map of F
21 F_inv = F.invert()
```

Listing 1. Defining and evaluating a Taylor Map in **sandalwood**

Map composition ( $H(x) = G(F(x))$ ) is computationally demanding as it requires substituting high-order polynomials into one another. **sandalwood** optimizes this by marshaling the entire map to the Fortran backend (when available) to leverage highly optimized, recursive substitution algorithms, or by utilizing Numba-JIT compiled dense multiplication tables in the native backend.

### V. AI AGENT INTEGRATION

As scientific discovery becomes increasingly automated, computational tools must be designed not only for human researchers but also for autonomous AI agents. **sandalwood**’s Pythonic API is uniquely positioned to serve as a high-fidelity environment for AI-driven scientific workflows, such as those governed by state-graph architectures. To natively support autonomous agents (such as **AIyaru**), we have implemented the **agent.tools** module. This module provides a complete integration layer that exposes the full breadth of the library’s capabilities via a highly optimized interface.

#### A. Stateful Session Registry and Context Optimization

A major limitation of utilizing Large Language Models (LLMs) for numerical and symbolic tasks is the context window length and tokenization overhead. A high-order Multivariate Taylor Function (MTF) or a vector-valued **TaylorMap** can contain thousands of floating-point coefficients. Serializing these structures into the LLM conversation history quickly saturates the context window and increases API costs.

To solve this problem, the **agent.tools** module implements a stateful Session Registry pattern (**registry.py**). Under this architecture, when an agent creates, parses,

or modifies a Differential Algebra (DA) object, the server registers the object and returns a lightweight JSON reference string containing a unique identifier (e.g., `{"ref": "mtf_0", "message": "..."}).` The agent subsequently references these short strings in future tool calls (such as partial differentiation, composition, or batch evaluation) without ever needing to see or parse the raw coefficients. The registry state can be inspected via read-only resources, optimizing the agent’s context and ensuring efficient execution.

## B. Standard Protocols and Tool Wrappers

To achieve compatibility across diverse agentic frameworks, the `agent_tools` module provides two primary interfaces:

- **Model Context Protocol (MCP) Server:** The module implements an MCP server (`mcp_server.py`) powered by FastMCP. It exposes 21 core differential algebra operations as standard MCP tools. It maps the session registry as a read-only resource (`registry://variables`), enabling LLMs to list and query registered variables on demand. It also features a specialized instruction template (`sandalwood-da-expert`) to guide the LLM’s reasoning.
- **LangChain Tools:** The library exposes the core functions wrapped in LangChain’s `@tool` decorator (`langchain_tools.py`) for out-of-the-box integration into LangGraph and other agentic graph frameworks.

## C. Robustness and Natural Language Parsing

AI agents frequently struggle with the precise array constructs and coordinate definitions required by numerical libraries. To bridge this gap, `sandalwood` provides a robust parser (`parser.py`) that enables agents to construct MTFs directly from natural mathematical strings (e.g., `"x1**2 + sin(x2)"`). The parser supports all standard trigonometric and hyperbolic functions, alongside specialized functions such as the inverse square root (`isqrt`) and specialized power series roots (`inv_cbrt`, `inv_pow_3_2`).

Furthermore, the core tool implementations (`functions.py`) enforce strict typing and are equipped with safe exception boundaries. Instead of throwing segmentation faults or silent NaN values, exceptions are caught internally and returned to the LLM as detailed JSON-serialized error strings. This allows the

autonomous agent to observe the failure, reason about the cause, and perform self-correction dynamically within its reasoning loop.

## VI. PERFORMANCE AND OPTIMIZATION

Achieving high performance in Python requires careful management of memory and loop execution. `sandalwood` introduces several critical optimizations:

- **Structure of Arrays (SoA) Layout:** Specialized physics kernels, such as the Biot-Savart integrator, enforce SoA memory layouts, vastly improving cache locality and SIMD vectorization potential during batch processing.
- **Parallel Evaluation:** Evaluating polynomials at millions of phase-space points is offloaded to OpenMP-parallelized Fortran kernels, resulting in speedups of up to  $100\times$  compared to standard NumPy broadcasting.
- **The CosyScope Context Manager:** When interfacing with the static memory pool of the Fortran backend, Python’s dynamic garbage collector can cause severe fragmentation. `sandalwood` implements an  $O(1)$  memory allocation tracker, `CosyScope`, which operates like a compiler stack frame, instantly freeing thousands of intermediate variables upon exiting a calculation block.

Benchmark results for large-scale operations (e.g., 100,000 source segments in a Biot-Savart calculation) demonstrate that the SoA layout and Fast-Path JIT dispatch yield speedups exceeding  $400\times$  compared to naive object-oriented Python implementations.

## VII. CONCLUSION AND FUTURE WORK

`sandalwood` provides a modern, robust, and high-performance Python interface for Multivariate Taylor Functions and Differential Algebra. By successfully bridging the ease-of-use of Python with the raw computational power of Numba and Fortran, it democratizes access to high-order DA methods for a broad scientific audience. The library is particularly well-suited for HPC applications in electromagnetics, beam physics, and celestial mechanics, while remaining fully compatible with automated, AI-driven discovery pipelines.

Future open-source development will focus on expanding native GPU acceleration (e.g., via CUDA and JAX), implementing advanced symplectic integration routines natively in Python, and further refining the AI-agent integration capabilities.

---

[1] J. F. Ritt, *Differential Algebra* (American Mathematical Society, 1950).

[2] M. Berz, “Differential algebraic foundations of nonlinear

- beam dynamics,” in *Nonlinear and Collective Phenomena in Beam Physics* (1999), pp. 1–42.
- [3] M. Berz, “COSY INFINITY—A new general-purpose optics code for the design of particle accelerators,” *Particle Accelerators* **24**, 109 (1989).
- [4] K. Makino and M. Berz, “COSY INFINITY Version 9,” in *Proceedings of ICAP 2006* (2006), p. 229.